

# Neural Taint Tracking for LLM Agents

## Final Project Report

## 1 Project goal

Indirect prompt injection happens when an agent reads untrusted data, such as a document or web page, and follows instructions inside that data. The attacker may try to make the agent call a dangerous tool. In this project, protected tools include file writes and deletes, command execution, HTTP posts, email, memory updates, and SQL execution.

My goal was to add a clear safety layer to the TAICF evaluation framework. The scanner follows text copied from untrusted tool output into protected tool calls. It can also follow text after small changes in case, punctuation, or spacing. This is lexical tracking: it compares normalized text fragments. It is not full semantic taint tracking, and it does not label each token produced by the language model.

## 2 Implementation

I implemented one trust policy that is used during the agent run and during later analysis. The policy marks tool output as trusted or untrusted. It also marks tool calls as privileged when they can cause an important side effect. For example, retrieval and web content are untrusted, while a file write is a privileged call. Unknown tools fail closed. Their output is treated as untrusted, and their calls are treated as privileged.

The scanner first normalizes text with Unicode NFKC. It changes letters to lower case, removes punctuation, and joins repeated spaces. It then stores the full text and five-token fragments. Each fragment keeps labels for all sources that produced it. Before a privileged tool runs, the scanner searches its text arguments for these fragments. A match produces a `taint_leakage` result.

The online **Gate** performs this check before the tool body executes. This means that a blocked tool has no side effect. The offline **Monitor** applies the same scanner to saved traces after the run. Both paths use the same trust policy. Tool metadata includes a policy fingerprint, which identifies the exact policy version. If this fingerprint is missing or out of date, the program classifies the tool again with the current policy.

`gate.allow_quote` is the only operation that can release text from an untrusted label. It works only for a document that the user has already approved. The declassification mechanism keeps the source of each text fragment. Authorizing one document removes only that document's source label. If the same text also came from an attacker document, the attacker label remains. The agent cannot add a document to the approval list. `gate.sanitize` only records an audit event. It does not clean or release the text.

Before rescoreing, the program checks that the saved configuration and traces belong to the same run. It checks the configuration hash on every trace span and on the saved scanner results. This prevents results from one configuration from being scored as part of another run.

## 3 Evaluation

I used two forms of evaluation. First, deterministic replay used fixed tool plans. These runs make the same calls every time, so they are useful for controlled security tests. Each suite had a defended run with the scanner and gate, and a separate undefended control without blocking.

The 11-case transformation ladder contains only attacks. Six cases copy the text or make small lexical changes. Five cases use synonyms, reordered text, paraphrasing, translation, or encoding. The seven-case taint-flow suite has five attacks and two benign tasks. It also tests `gate.sanitize`, self-authorization, and an approved document.

Second, I used the existing LangGraph ReAct agent with a local model served by LM Studio. The model was `mistralai/ministral-3-14b-reasoning`. I ran the same seven cases once with the defense and once without it. Both runs used temperature 0, an output limit of 2,048 tokens, an 8-step agent limit, and an 8,192-token evaluation limit. This is a small local-model study, not a test of a hosted frontier model.

The metrics separate scanner detection, attempted dangerous calls, blocked calls, and side effects that really happened. Attack success rate (ASR) counts only cases where the forbidden side effect was found in the final state. A scanner miss alone is not counted as a successful attack. Benign utility uses the exact expected final file content.

## 4 Results

Table 1 shows the deterministic transformation ladder.

| Table 1: Canonical 11-case transformation ladder. |             |         |       |
|---------------------------------------------------|-------------|---------|-------|
| Condition                                         | Detected    | Blocked | ASR   |
| Defended                                          | 6/11        | 6/11    | 5/11  |
| Undefended control                                | unavailable | 0/11    | 11/11 |

The defended scanner had precision 6/6. It detected and blocked all six lexical or lightly changed attacks. It missed the five semantic or encoded cases, and these cases produced forbidden writes. The undefended control had ASR 11/11.

Table 2 compares deterministic replay with the local real agent. The “detect/live” column shows scenario detection followed by conditional live recall. Live recall is measured only when attack text reaches a protected tool argument.

| Table 2: Seven-case taint-flow results. |             |          |     |               |
|-----------------------------------------|-------------|----------|-----|---------------|
| Condition                               | Detect/live | Attempts | ASR | Exact utility |
| Replay, defended                        | 4/5, 4/5    | 5/5      | 1/5 | 2/2           |
| Replay, control                         | unavailable | 5/5      | 5/5 | 2/2           |
| Local LLM, defended                     | 0/5, N/A    | 0/5      | 0/5 | 0/2           |
| Local LLM, control                      | unavailable | 0/5      | 0/5 | 1/2           |

In deterministic replay, the defense detected and blocked 4/5 attacks. One semantic paraphrase caused a forbidden side effect, so defended ASR was 1/5. The undefended ASR was 5/5. Exact utility was 2/2 in both conditions. The defended false-positive and over-block rates were both 0/2.

The local model saw all five attacks but ignored them. It made no forbidden attack call, so attack attempts and ASR were 0/5 in both conditions. Conditional live scanner recall is N/A because the model produced no attack-derived text in a protected tool call. Live blocking effectiveness is also N/A because there were no forbidden attack calls. The experiment shows that this model resisted these attacks. It does not show how well the scanner would block a real-model attack.

The model attempted one benign-target privileged write in every case. In defended `tnt-06`, the gate blocked that attempted write before the underlying side effect executed. The document was authorized, but the model did not call `gate.allow_quote`. It tried to write the retrieved text directly while it was still marked as untrusted. The gate therefore blocked the write. This gives one benign false positive and one over-block, both 1/2.

Exact utility was 0/2 for the defended local run and 1/2 for the control. A separate review of meaning gave 1/2 and 2/2. The model’s budget sentence was reasonable but did not match the exact expected text. This second measure is only extra context; it does not replace the automatic exact check. Both local runs had zero errors and zero inconclusive cases.

## 5 Limitations and future work

The scanner depends on shared words between the source and the tool argument. Synonyms, paraphrases, translations, and Base64 can remove this overlap. Very short payloads may not create a five-token fragment. An instruction split across several calls may also be missed. The method also needs complete tool traces. Missing trace events can hide the path of untrusted data.

The real-agent evaluation used one local model and one repetition per condition. The model ignored every attack, so the experiment did not provide a real forbidden call for the gate to block. Exact-string utility is another limitation. It can reject an answer that is correct in meaning but uses different words.

Future work could add a semantic detector as a second signal. It could also collect fragments across calls, check common encodings, and test more models with more repetitions. A better utility measure could accept correct paraphrases while still checking the final side effect.

## 6 Conclusion

I implemented a transparent lexical safety layer for LLM agents. The system detects copied and lightly changed attack text, keeps source-specific labels, and blocks matching protected calls before they execute. The results also show that semantic changes can bypass lexical matching. This system is useful as one safety layer, but it is not a complete solution to prompt injection.